Business Banking RM Intelligence Hub

Project report and AI assisted development workflow

Project owner nandinikawale | 11 October 2026

We built a working business banking dashboard and a separate, reproducible synthetic data generator. This report explains the application, the seven datasets, the prompts that guided development, the iterations we tried, and observations from using Codex in VS Code.

Business purpose

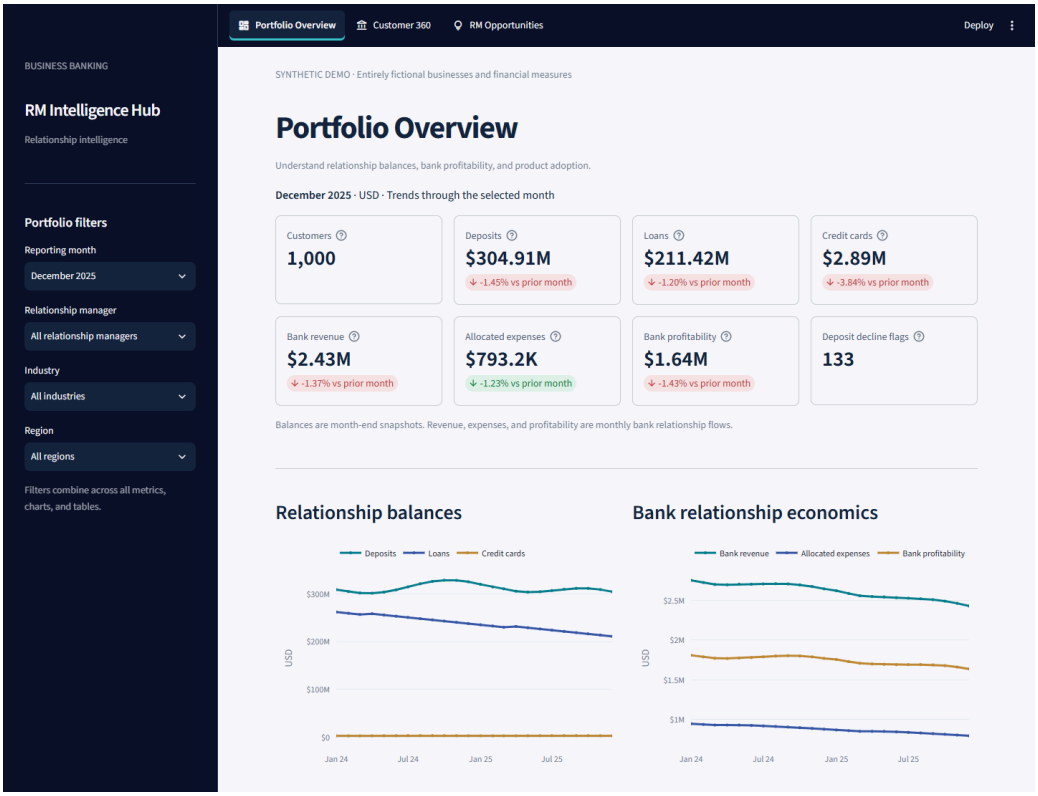
A relationship manager coordinates business customer relationships, understands customer needs, and works with credit and product specialists. The hub supports portfolio reviews, meeting preparation, and conversations about deposit declines or potential service gaps. These are potential benefits; revenue growth, retention improvement, and time savings have not been measured.

What the application provides

Portfolio Overview summarizes balances, monthly bank profitability, product adoption, and concentration. Customer 360 brings together a fictional business profile, financial history, holdings, and signals. RM Opportunities presents explainable retention and cross-sell indicators, including separate signal and unique-customer counts.

Technical design

Streamlit presents the interface; DuckDB queries local CSV data; Pandas handles tabular processing; Plotly renders charts. Faker and NumPy support fictional profiles and financial simulation. uv manages the environment and locked dependencies. Generation lives in src/banking_data, analytics in src/rm_hub, and presentation in streamlit_app.



Current application appearance with the default December 2025 synthetic portfolio.

Synthetic datasets and generation

The default dataset contains 1,000 fictional business customers, 20 fictional relationship managers, and 24 months from January 2024 through December 2025. Seed 42, stable ordering, explicit CSV formatting, and locked dependencies support reproducibility. Currency is USD.

CSV file	Rows	Purpose
customers.csv	1,000	Fictional business profiles and RM assignments
relationship_managers.csv	20	Fictional RM profiles and regions
deposits.csv	24,000	Deposit balances, cash movements, business sales, expenses and profit
loans.csv	24,000	Holdings, balances, drawdowns, repayments and interest revenue
credit_cards.csv	24,000	Holdings, purchases, payments, limits and bank revenues
profitability.csv	24,000	Bank revenue components, allocated costs and profitability
products.csv	24,000	Monthly product holdings and demonstration cross-sell flags

Connected financial measures

The same customer_id links profiles and all customer-level tables; monthly joins also use month. Loan and card nonholders retain monthly rows with zero balances and product income. Deposit liquidity scales with business sales, while industry margins, seasonality, and persistent shocks make histories more structured than independent random values.

Business operating profit equals business sales minus operating expenses. Bank relationship profitability is a separate measure: total bank revenue minus allocated expenses. Average deposits influence deposit revenue through an invented spread; average loans and interest rates influence lending revenue. Deposit interest expense is already netted in the deposit component and is not deducted twice.

Scenarios and explainable indicators

The simulation includes stable, growth, seasonal, attrition, and distressed scenarios. Scenario labels are generation diagnostics, not inputs used to prioritize customers in the app. A retention indicator requires three consecutive deposit declines, at least 20% cumulative decline, and a starting balance of at least USD 10,000. Cross-sell rules combine a missing product with fictional activity thresholds.

Standalone generation and bundled demo data

The seven default CSVs are included in GitHub so a fresh clone can run the app after dependency installation. Regeneration is optional and runs independently of Streamlit. It overwrites the default CSVs and writes local validation and hash reports; alternative datasets and generated reports remain ignored.

No real customer data, identifiers, public economic datasets, internal banking datasets, or confidential rules are used. An early request contemplated public economic supplementation; the later instruction to use only fictional data determined the final implementation.

Prompts that guided development

The following excerpts are quoted from the project conversation. They illustrate how requirements, planning requests, and corrections shaped the build; they are a selection, not a complete prompt transcript.

Set the engineering role

"Act as a senior Python and Streamlit engineer."

The initial request established Streamlit, uv, DuckDB, Pandas, Plotly, Faker, three pages, reusable functions, and separation of data, business logic, and presentation.

Separate generation from the app

"Keep the synthetic data generation separate from your Streamlit application. I need to upload this project to Github as well."

This led to a standalone generator and CLI, a modular application, and repository documentation rather than generation embedded in page rendering.

Require reproducibility

"Only create data for fixed dates from Jan 2024 to Dec 2025 and use a constant random seed."

Fixed dates and seed 42 replaced time-dependent generation. The requested customer count was reduced from approximately 5,000 to 1,000.

Plan before executing

"Let's move on to the next step. Do not execute anything yet. SHare the plan with steps and details of what you will be performing in order to implement it and its function."

Plans and command explanations provided review points before implementation. Follow-up approvals then authorized the next stage.

Add business context

"Can we add some details about how the busienss banking business works and what a relationship manager is expected to do and hwo this project can help the business?"

A concise README overview and docs/business_context.md explain the business purpose, RM workflows, financial definitions, and limits.

Evaluate and reverse a design decision

"revert back these changes to the previous version. I like the older version better."

The slate blue theme and enlarged branding were reverted. The retained design uses midnight navy navigation, teal highlights, a light dashboard, and the original smaller caption.

Other prompts asked for consistent IDs, financial reconciliation, realistic attrition histories, each file's purpose, clone instructions, and explanations of terminal commands. These constraints were as important as the initial feature request.

Iterations and decisions

Scope and data realism

We planned the repository before building the pages. The project was renamed business-banking-rm-intelligence-hub and placed in the requested repository folder. The default scope became 1,000 customers, seven CSVs, fixed dates, and a constant seed. Later requirements excluded all real-world data.

Generation focused on relationships across financial measures and meaningful changes over time. Validation checked IDs, row counts, holdings, reconciliation, and indicator logic. This gave the application a consistent foundation rather than seven unrelated files.

Analytics and the three pages

After generation, we added local DuckDB analytics and built Portfolio Overview, Customer 360, and RM Opportunities in stages. Business calculations were kept outside presentation code. Historical views use observations through the selected month; later data and generator scenario labels are excluded.

Navigation and appearance

The initial sidebar page menu changed to native horizontal navigation on desktop, while filters remained in the sidebar. At phone widths, Streamlit exposes the page links through its responsive sidebar menu. Browser checks covered desktop placement and mobile navigation.

We reviewed an AI-generated concept image, then implemented midnight navy navigation and captured the actual app for the README. We subsequently tried slate blue and an 18 px semibold BUSINESS BANKING label. The browser caught a font-size override, which was corrected. After reviewing the result, the user preferred the earlier design, so both changes were reverted.

Making the repository easier to run

Initially, generated CSVs were ignored and clone instructions required generation before launch. We changed this so the seven default fictional CSVs are committed and regeneration is optional. The ignore rules and ZIP packaging include only that default dataset; alternative data and reports stay excluded.

The README gained all three page descriptions, setup and launch commands, troubleshooting, and business context. We added contribution guidance, publication instructions, an MIT license, line-ending rules, and a GitHub Actions workflow. The user uploaded the repository to GitHub.

Environment troubleshooting

A VS Code terminal tried to run uv through the project's virtual-environment Python, where uv was not installed. Documentation now provides the Windows launcher form, py -3.13 -m uv, and direct virtual-environment app launch instructions. The workflow also clarified Git's staged-file summary viewer: pressing q exits the viewer before the next command.

The project remains a local demonstration. Uploading the source to GitHub does not by itself host a live application.

Learnings and final result

What the workflow showed

Specific domain constraints produced a more credible demo. Requiring shared IDs, linked balances and income, reconciliation, and sustained decline scenarios made the data useful for explaining business questions.

Separating generation, analytics, and presentation made changes easier to isolate. Including default data improved the first-run experience without removing the ability to reproduce it. Fixed dates and a seed made regeneration assessable rather than dependent on the day a user cloned the repository.

Incremental planning created useful review points. The user chose scope, navigation, licensing, and appearance; Codex helped implement code, documentation, tests, and checks. Short approvals worked when the preceding plan already defined the change, while explicit constraints guided larger decisions.

AI-generated visuals were useful for exploring appearance, but the final README uses an actual app screenshot. Browser verification mattered: it caught a font-size override that code linting could not detect. The later design reversal also showed why changes should stay easy to undo.

Documentation was part of usability. Clone instructions, the uv interpreter explanation, business context, and optional regeneration reduced the knowledge needed to try the application.

Verification and interpretation

A fresh-install package check passed all 51 tests, linting, formatting, data validation, and an analytics smoke check. Regenerated CSV hashes matched the original default data in the same locked environment. Subsequent styling checks passed 21 presentation tests and verified browser rendering and mobile navigation.

These are local development results. A GitHub Actions workflow is configured, but its remote run status is not asserted here. Synthetic indicators do not establish actual attrition, product suitability, or credit approval. The app does not implement underwriting or outreach tracking, and business impact has not been measured.

Run the completed application

From the project root on Windows, install the dependencies and launch. No data generation is required for the included default dataset.

```
py -3.13 -m pip install uv==0.12.24
py -3.13 -m uv sync --locked --group app
py -3.13 -m uv run --locked --group app streamlit run streamlit_app/app.py
```

Open the local URL printed by Streamlit. Optional regeneration runs separately:

```
py -3.13 -m uv run --locked banking-data generate
py -3.13 -m uv run --locked banking-data validate
```

Repository and supporting documentation

https://github.com/nandinikawale/business-banking-rm-intelligence-hub

License: MIT. Supporting files: README.md, data/README.md, docs/business_context.md, docs/architecture.md, docs/data_dictionary.md, and docs/simulation_assumptions.md.